1. Multitarea

A menos que tengamos un código que sólo cumpla una función, habitualmente tendremos diversas tareas independientes que se tienen que completar en diferentes momentos, y que potencialmente intercambian información entre ellas. A grandes rasgos, se distinguen dos modelos de multitarea:

1. En un sistema guiado por **eventos**, existe una colección de funciones asociadas a un conjunto de eventos externos. La ocurrencia (asíncrona) de eventos determina el secuenciamiento de las funciones, que en caso de eventos simultáneos se ejecutan según prioridad. Los eventos pueden ser detectados mediante *polling* o mediante interrupciones.
2. En un sistema guiado por **tiempo**, las funciones se asocian con un conjunto de instantes de tiempo. En este caso, el transcurso (síncrono) del tiempo es quien determina el secuenciamiento de funciones, también con un atributo de prioridad para casos de ejecuciones simultáneas. El paso del tiempo se mide utilizando un temporizador, que interrumpe con una cierta periodicidad (ticker del sistema).

1.1. Arquitecturas multitarea cooperativa

Existen numerosas arquitecturas para la ejecución multitarea, que generalmente encuentran un compromiso entre simplicidad y capacidad.

1.1.1. Super-loop

Un único bucle infinito comprueba, en orden cíclico, el estado de los dispositivos de entrada/salida, y los sirve cuando sea necesario. En este modelo no hay interrupciones. El problema es que las latencias o tiempoes de respuesta son impredecibles, y en el peor caso implican la duración completa de una vuelta al bucle. Además, la arquitectura es frágil pues cualquier cambio en el código altera los tiempos, y el comportamiento del sistema es muy dependiente de cómo esté programada la secuencia de eventos. El esquema básico de Arduino, por ejemplo, funciona con esta arquitectura:

```
void init() {
    //configuración de registros, relojes, etc
    loop();
}

void loop() {
    while (1) {
        if (task1_ready)
            task1();
        if (task2_ready)
            task2();
        //...
    }
}
```

1.1.2. Foreground/Background

En esencia, un *super-loop* con interrupciones. Varias tareas atienden a los dispositivos de E/S, implementadas como RTI. Estas tareas únicamente marcan eventos, actualizan variables y leen o escriben datos en colas. Se ejecutan en *foreground*, ganando prioridad sobre la tarea principal, y pueden o no anidarse. La tarea principal, siempre en background, realiza un bucle infinito y tiene la mínima prioridad. Irá procesando, poco a poco (siguiendo la estructura del super-loop) las diversas variables que se hayan disparado en las RTI.

Tiene la problemática de ubicar el grueso del procesamiento de los dispositivos de E/S. Si se coloca este procesamiento en *background*, todos los dispositivos acabarían de facto teniendo la misma prioridad, con retrasos similares a super-loop en el peor de los casos. Si se hace en *foregrond*, las RTI se alargan y se penaliza el tiempo de respuesta.

```
void init() {
    //configuración de registros, relojes, etc
    //configuración de RTI
    loop();
}

void loop() {
    while (1) {
        if (flag_a) {
            flag_a = 0;
            task_a();
        }
        if (flag_b) {
            flag_b = 0;
            task_b();
        }
        //...
    }
}

void isr_a_handler() {
    //trata isr de periférico a
    flag_a = 1;
}

void isr_b_handler() {
    //trata isr de periférico b
    flag_b = 1;
}
```

1.1.3. Cola de funciones

Añade prioridad a la ejecución de funciones *foreground/background* mediante una pila de tareas pendientes. Esta pila se llena al ser disparadas las interrupciones. La tarea principal *main* está continuamente de nuevas tareas apareciendo en la pila, y las ejecuta según llegan, respetando por tanto el orden de interrupción, pero no necesariamente de prioridad en caso de que una tarea más prioritaria interrumpiera tras una menos prioritaria.

```
volatile queue_t queue;

void init() {
    //configuración de registros, relojes, etc
    //configuración de RTI
```

```
    loop();
}

void loop() {
    while (1) {
        if (!queue.empty()) {
            ptr_task = queue.dequeue();
            (*ptr_task)();
        }
    }
}

void isr_a_handler() {
    //trata isr de periférico a
    if (!queue.full())
        queue.enqueue(task_a);
}

void isr_b_handler() {
    //trata isr de periférico b
    if (!queue.full())
        queue.enqueue(task_b);
}
```

1.2. Arquitecturas RTOS

En las diferentes arquitecturas de multitarea cooperativa, se presenta un problema común: cuando una tarea está en ejecución y se bloquea, siempre bloqueará al resto. Esto presenta un gran problema pues el sistema completo se cae debido a un único punto de fallo. Para solventarlo, se debe dejar de confiar en el aspecto *cooperativo*, y moverse hacia una arquitectura *expropiativa* capaz de retirar de ejecución las tareas. Esto se hace mediante la presencia de un Sistema Operativo en Tiempo Real o **RTOS**.

En él, varias tareas RTI atienden a los dispositivos de E/S en *foreground*, mientras que otras tareas en *background* realizan el resto de funciones, ya sean independientes o disparadas en algún momento por las RTI. No hay necesariamente tarea principal o *main*, que desaparece tras configurarse todo el sistema. Para comunicarse entre sí, las tareas utilizan variables compartidas, lo cual implica la necesidad de poder detectar y resolver el acceso a secciones críticas.

Existe un código de **kernel** que se encarga de orquestrar todas estas intercomunicaciones. Ofrece mecanismos para sincronizar y comunicar tareas, decide según prioridades la tarea en ejecución, y lo más importante, **conmuta** la ejecución de tareas dinámicamente. La única desventaja presente en este sistema es la sobrecarga de tener al propio kernel en ejecución, pero sus ventajas lo compensan ampliamente.

Aunque un RTOS puede ser cooperativo (donde una tarea cede voluntariamente el control al kernel, terminando o mediante señales específicas), lo más habitual es contar con un RTOS expropiativo, donde las excepciones (RTI o propias del kernel) disparan la conmutación de tareas. En este caso, ante los eventos:

- La tarea en ejecución se interrumpe y el kernel guarda su contexto.
- La RTI se ejecuta y potencialmente pide al kernel la ejecución de cierta tarea.
- El kernel marca como preparada la ejecución de la tarea que esperaba el dato.

- El planificador conmuta a la tarea más prioritaria en cada momento, que podría ser (o no) la nueva tarea encolada.

```
void main() {
    // crea tareas
    // initializa RTOS
}

void isr_a_handler() {
    //trata isr de periférico a
    signal(A);
}

void isr_b_handler() {
    //trata isr de periférico b
    signal(B);
}

void task_a() {
    while (1) {
        wait(A);
        fun_a();
    }
}

void task_b() {
    while (1) {
        wait(B);
        fun_b();
    }
}
```

Para la sincronización entre las diferentes tareas (en *foreground* o *background*) se cuenta con mecanismos ofrecidos por el kernel como semáforos, mutex, variables compartidas... propios de un sistema operativo.

Cuando los datos se comparten entre diversas tareas, existe el problema de la desincronización cuando, por ejemplo, una tarea es interrumpida mientras está consultando los datos proporcionados por otra:

```
volatile uint16_t temp[2];
void leerTemperatura_isr(void) {
    // borraflag de int. pendiente
    temp[0] = // ...leer del HW...
    temp[1] = // ...leer del HW...
}

void main(void) {
    uint16_t0, t1;
    while(1) {
        // <--- INT: La RTI lee dos valores iguales (7 y 7 por ejemplo)
        t0 = temp[0];
        // <--- INT: La RTI lee dos valores iguales (8 y 8 por ejemplo)
        t1 = temp[1];
        // Sin embargo, la alarma suena al haberse guardado en momentos
            diferentes
        if( t0 != t1 )
            //suena alarma
    }
}
```

La solución pasa por hacer accesos atómicos a los datos compartidos, deshabilitando las interrupciones al acceder a los datos:

```
while(1) {
    INT_DISABLE();
    t0 = temp[0];
    t1 = temp[1];
    INT_ENABLE();
    if( t0 != t1 )
        //suena alarma
}
```

Aún así, este mecanismo no es trivial. Hay que asegurarse que toda inactivación es seguida por una activación, y que no se anidan parejas de inactivación/activación, ya que podrían activarse accidentalmente antes de tiempo.

Las interrupciones son, con todo, la forma idónea de dar respuesta rápida a eventos. Su latencia depende de:

1. El tiempo máximo en que están deshabilitadas
2. El tiempo que tarda en ejecutarse cualquier RTI de mayor prioridad (en caso de interrupciones anidables).
3. El tiempo que tarda el procesador en detectar la interrupción y saltar a la RTI.
4. El tiempo que se tarda en guardar el contexto y realizar la acción de respuesta.

El código por tanto deberá ser pequeño y eficiente para reducir los tiempos de ejecución y cambio de contexto, las secciones críticas cortas para reducir el tiempo de deshabilitación, y la arquitectura eficiente para detectar y saltar rápido.

1.3. Correción en aplicaciones multihebra

En una aplicación multihebra (por hebra entendemos un hilo de ejecución, ya sean tareas, interrupciones, etc), la correción debe garantizarse para evitar comportamientos erróneos (bloqueos principalmente).

Por un lado, los accesos a secciones críticas (zonas del código donde 2 o más hebras ueden acceder a la vez a un recurso compartido) deben ser atómicos. Es decir, sólo una de las dos puede estar en esa zona del código a la vez, para evitar que los valores se puedan consultar o escribir de manera errónea. Además, una vez se comienza un código atómico, se debe poder completar sin interrupción de otra hebra que acceda al mismo recurso.

Por otro lado, el código debe diseñarse para ser *reentrante*, esto es, que las variables locales se almacenen en pila y las globales se accedan de manera atómica, permitiendo así la ejecución por parte de varias hebra del mismo código.

Las secciones críticas siguen 3 principales patrones:

- *Read-modify-write*: Se lee un valor que se copia localmente, se modifica, y finalmente se actualiza la variable global que lo contiene.

- *Write-read*: Se escribe primero un valor, y se lee esperando que siga presente.
- *Multistep-write*: Se realizan varias escrituras consecutivas a memoria para valores que con una única instrucción no se actualizan (por ejemplo, estructuras complejas).

En arquitecturas super-loop, por ejemplo, no hay secciones críticas, pues solo existe una hebra. En arquitecturas foreground/background la hebra principal o `main` deberá proteger las secciones críticas inhabilitando las interrupciones. En un RTOS cooperativo, el programador controla cuándo cambia el contexto y por tanto podrá proteger las secciones críticas. En un RTOS expropiativo, no se tiene el control sobre los cambios de contexto y todas las hebras deberán proteger las secciones críticas de manera homogénea.

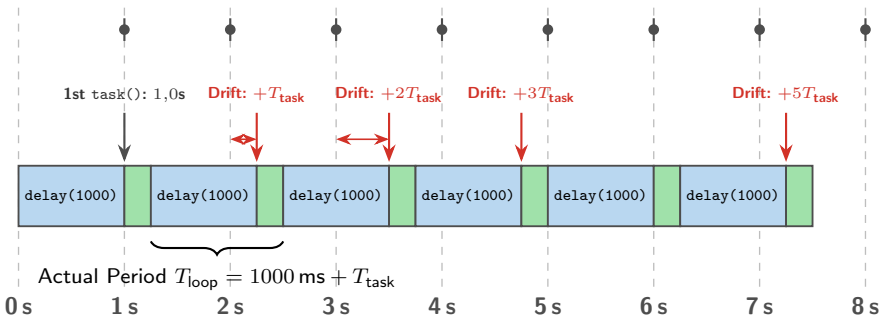
1.4. Sistemas guiados por tiempo

Existen gran número de sistemas empotrados que deben soportar la llamada periódica a determinadas funciones. Por ejemplo al refrescar un display a un cierto número de *frames* por segundo, muestrear sensores a una frecuencia concreta, etc. También es común que se realicen secuencias de acciones con cierta temporización, como por ejemplo en sistemas complejos que deben abrir/cerrar válvulas, mover motores, etc.

1.4.1. Primera aproximación

Una aproximación mediante sistemas *super-loop* nos llevaría al siguiente modelo de temporización, que falla por motivos obvios al no tenerse en cuenta el propio tiempo de ejecución de la tarea para separar las ejecuciones. Además, la CPU está activa todo el tiempo al realizar esperas activas:

```
void task_init( void );
void task( void );
void delay( uint16 ms );
void main( void ) {
    sys_init();
    task_init();
    while( 1 ) {
        delay_ms( 1000 );
        task();
    };
}
```



1.4.2. Segunda aproximación

Otra aproximación, que podría mejorar a la anterior, y evita tener a la CPU dormida, es tener un timer que se dispare cada cierto tiempo. Por ejemplo:

```
timer0_isr() {
    // borrar bits de ISR pendiente
    // no hace nada más, sólo saca del sleep();
}

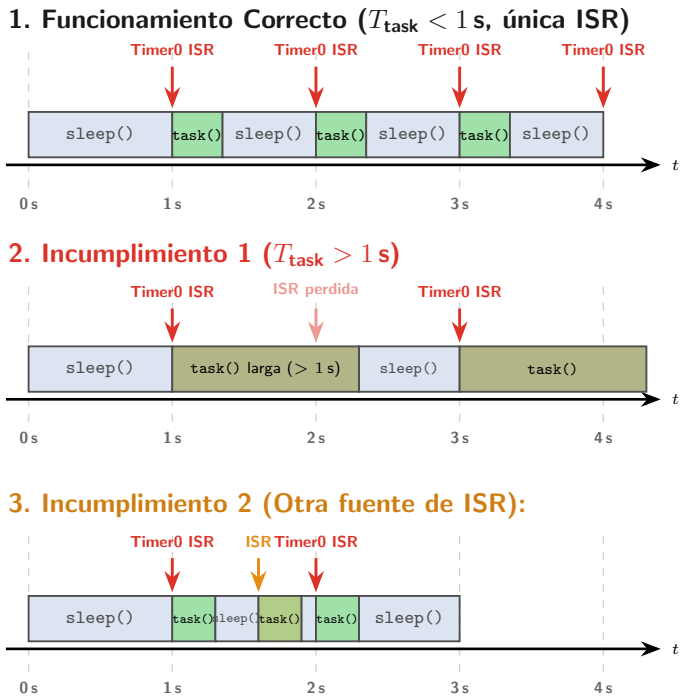
void main(void) {
    SET_TIMER_MS(timer0, 1000);

    while( 1 ) {
        sleep(); //entra en modo de sleep, se despierta con ISR
        task();  //realiza la tarea
    }
}
```

Esta aproximación funcionará mientras:

1. El tiempo de `task()` sea inferior siempre a 1 segundo.
2. Sólo exista una única fuente de interrupción.

Si se inclumple la primera, la tarea no se disparará el número de veces adecuado, saltando ciclos. Si se incumple la segunda, el problema vendrá porque otras fuentes de interrupción hagan que la llamada a `sleep()` acabe antes de tiempo, disparándose en más momentos que sólo cada segundo.



1.4.3. Tercera aproximación

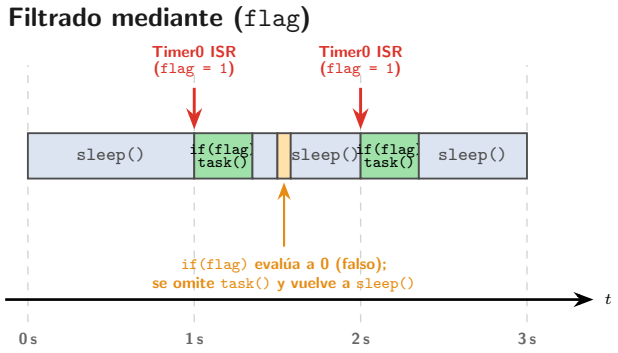
Una opción que sí sería funcional implicaría ejecutar la tarea en la propia ISR. Esto no es del todo recomendable, ya que las ISRs se deben mantener lo más escuetas posibles. Aunque no podemos solucionar el problema de que la tarea dure más de un segundo, sí podemos resolver el problema de que se lancen varias ISRs diferentes, mediante la inclusión de un flag que nos indique, al despertar del `sleep()`, si queremos de verdad ejecutar la tarea.

```
uint32_t flag = 0;

timer0_isr() {
    // borrar bits de ISR pendiente
    flag = 1;
}

void main(void) {
    SET_TIMER_MS(timer0, 1000);

    while( 1 ) {
        sleep(); //entra en modo de sleep, se despierta con ISR
        if (flag) {     //si flag está a 0, volverá a sleep()
            flag = 0;   //marca flag a 0
            task();     //realiza la tarea
        }
    }
}
```



Con este sistema, se pueden tener incluso varias tareas, disparadas por varios timers. También que un mismo timer, en función de una cuenta del número de disparos, active uno u otro flag para diferentes tareas. Mientras los tiempos de ejecución sean deterministas, e inferiores al periodo de disparo de el/los timer/s, cabrán en los mismos y el código funcionará correctamente.

1.4.4. Polling con espera en tareas periódicas

Una cuestión importante, y que hay que evitar en cualquier tipo de ejecución de tareas periódicas, es el polling con espera activa (por ejemplo esperando a que un periférico esté activo). Ya que, si el periférico nunca se activa, o tiene un error, nos quedaremos bloqueados esperando a que devuelva un control que nunca volverá.

Para solucionar este problema, se puede utilizar polling con timeout (dando un error tras un cierto tiempo), polling con consulta (si no está disponible, devuelve inmediatamente un error),

o un polling ciego (sin comprobación) si se tiene la certeza de que el dato está disponible (por ejemplo en sistemas sincronizados, aunque no es recomendable).

1.5. Planificadores cooperativos

Cuando se dispone de múltiples tareas, una primera aproximación puede incluir el preprogramar su orden de ejecución mediante una lista que se ejecuta periódicamente. Sin embargo, estos sistemas (llamados *cyclic-executive*) son muy poco flexibles, especialmente cuando se interactúa con personas o entornos que pueden generar interrupciones en momentos arbitrarios.

Como solución, se introduce el concepto de planificadores. En las Apartados 1.1 y 1.2 hemos visto diferentes ideas de cómo varias tareas pueden ejecutarse, pero no hemos explicado el concepto de prioridades entre ellas, y cómo un planificador puede (y debe) evaluar entre las tareas disponibles, cuál se debería ejecutar (basándose en prioridades, deadlines de ejecución, etc).

En todo momento existe una úncia tarea en ejecución, y se ejecutan hasta completarse, sin ser interrumpidas por otras. Las tareas se preparan con periodicidad fija (p.ej disparándose con un timer), se encolan en una lista de tareas preparadas y, cuando la CPU está libre, elige entre las mismas siguiendo algún criterio (puede ser simplemente el orden de cola, o algo más complejo basándose en prioridades).

Se necesita una fuente de interrupción periódica que dispara la ejecución de las tareas y, opcionalmente, pueden existir otras interrupciones que, en caso de poder ejecutarse simultáneamente, deberán proteger sus recursos mediante exclusión mutua.

Este tipo de planificador es muy simple, y la ejecución de tareas es predecible y fiable. Los recursos habitualmente no serán compartidos, por lo que no suele haber muchas secciones críticas. Sin embargo, tareas poco prioritarias, o eventos externos, pueden sufrir retrasos grandes debido a que la única forma de cambiar de tarea, es que la tarea actual termine su ejecución.

Las tareas se suelen definir por una estructura como la siguiente, llamada bloque de control de tarea:

```
typedef struct {
    void (* pfunction) (void);  //puntero a función ejecutora
    uint32 period; //periodo entre 2 ejecuciones
    uint32 ticks;  //cuenta de ticks desde la última ejecución
    boolean ready; //1 si está lista para ejecución
} task_t;

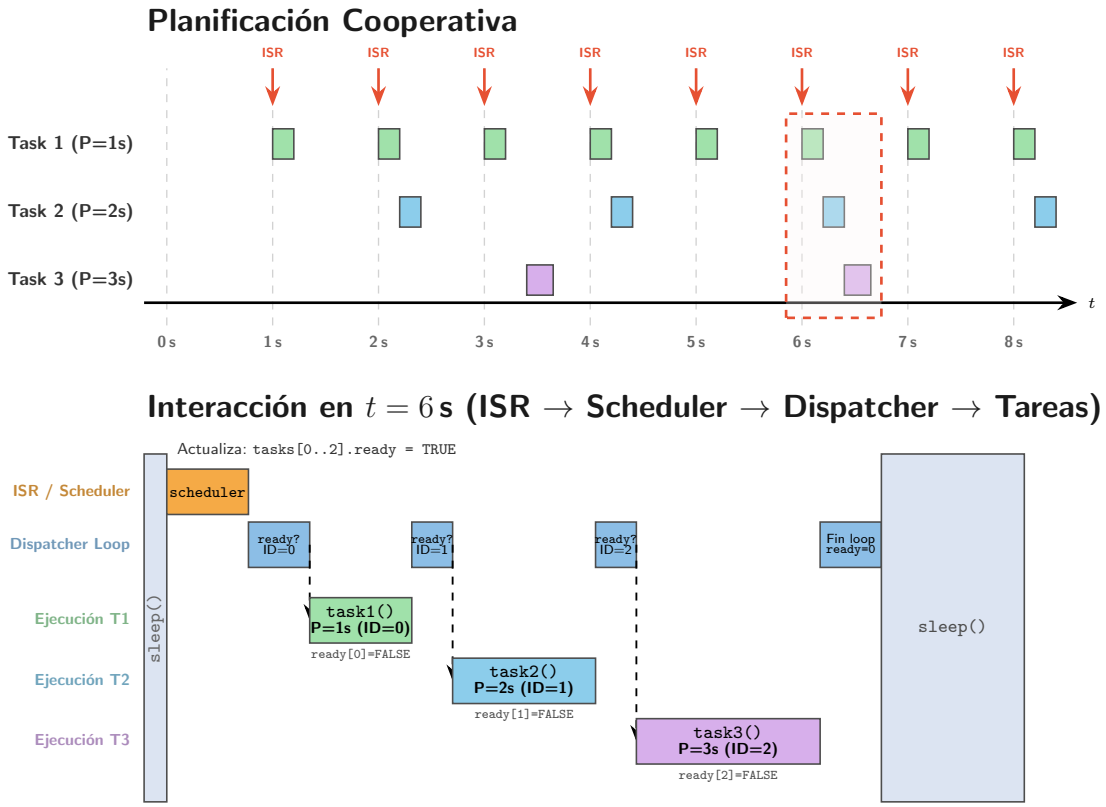
#define MAX_TASKS (10)
task_t tasks[MAX_TASKS];
```

A su vez, la gestión se divide en un planificador (que marca las tareas como listas), y un despachador o lanzador que las ejecuta:

```
//la rutina del timer es quien, de cuando en cuando,
//evalúa la planificación de tareas
timer0_isr() {
    // borrar bits de ISR pendiente
    scheduler();
}

void scheduler (void) {
```

```
9       uint32 id;
10      for (id=0; id<MAX_TASKS; id++)
11          if (tasks[id].pfunction)
12              if (++tasks[id].ticks == tasks[id].period) {
13                  //si una tarea llega a su periodo, se marca como READY
14                  //pero nunca expropia a otra (tras la ISR se volverá
15                  //a la ejecución en curso)
16                  tasks[id].ticks = 0;
17                  tasks[id].ready = TRUE;
18              };
19  }
20
21  void dispacher( void ) {
22      uint32 id;
23      for (id=0; id<MAX_TASKS; id++)
24          //el dispatcher ejecuta las tareas que están
25          //ready, una tras otra
26          if (tasks[id].ready == TRUE) {
27              (*tasks[id].pfunction)();
28              tasks[id].ready = FALSE;
29          }
30  }
31
32  void main (void) {
33      sys_init();
34      scheduler_init();
35
36      TIMER_INIT(timer0, 1000);
37      while (1) {
38          if (!tasks_ready()) //duerme hasta que interrumpa el scheduler
39              sleep();
40          //tras lo cual puede haber tareas listas para despachar
41          dispacher();
42          //si no las hubiera, o tras ejecutar, vuelvo a dormir
43      }
44  }
```



El modelo de planificador cooperativo se puede hacer más completo indicando, por ejemplo, prioridades, tiempos de activación de tareas, y multitud de parámetros adicionales para hacerlo más complejo. Cada uno tendrá sus ventajas e inconvenientes. Lo más importante es conocer que, en cualquier caso, la distribución óptima de tareas es un problema NP-complejo. Por muy inteligente que hagamos nuestro planificador, en sistemas con una gran contingencia entre tareas, lo más que podremos hacer es aproximarnos a una ejecución ideal de las tareas. Es por ello muy importante contar con las herramientas de perfilado adecuadas como para asegurar que el sistema cumple con los requisitos establecidos.

1.5.1. Bloqueos

Otro aspecto importante en este tipo de sistemas es la capacidad de recuperarse ante bloqueos. Si una tarea se queda atascada, el dispatcher no puede por sí mismo pararla. Para ello, se pueden introducir *watchdogs*. Un *watchdog* es un temporizador que, en caso de llegar a su límite, resetea el sistema a un cierto estado. Se puede, por ejemplo, en cada iteración del dispatcher, resetear el temporizador. Así, si se deja de resetear (por un bloqueo de alguna tarea) el temporizador llegará a su límite y reiniciará el sistema. La gestión se puede hacer de la siguiente manera:

```
1
2               void wd_isr() {
3                   SYSTEM_RESET();
4               }
5
6               static uint16 timeout;
7
8               void wd_on (uint16 dms) {
9                   timeout = dms;
10                  WTCNT = dms;
```

```
11                      //configuramos el watchdog para generar interrupciones al
                            llegar a
12                      //0 (cuenta descendente). Si llega, reiniciará el sistema
13                  }
14
15                  void wd_kick (void) {
16                      //reinicia la cuenta regresiva
17                      WTCNT = timeout;
18                  }
19
20                  void dispacher( void ) {
21                      //funcionalidad de dispatcher habitual
22                      //si se queda atascado, no llega nunca aquí,
23                      //y por tanto no reinicia el watchdog
24                      wd_kick();
25                      //también es posible crear una tarea wd_kick()
26                      //que vaya en la propia lista del dispatcher a máxima
                            prioridad
27                      //en caso de sistemas más complejos
28                  }
```

Al reiniciarse el sistema, se pueden consultar registros internos del procesador, o valores en memoria, para saber las causas (por ejemplo, si así lo preparamos y programamos, conocer la tarea causante del bloqueo).